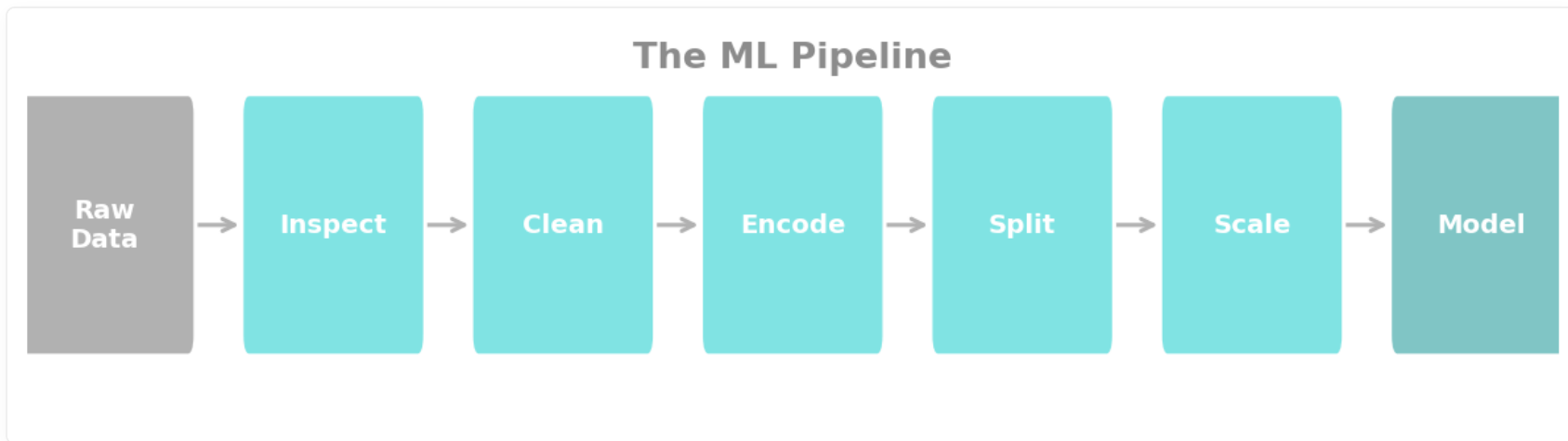


Data Selection, Cleaning & Preparing

Applied Machine Learning — Session 1, Chapter 2

The ML Pipeline



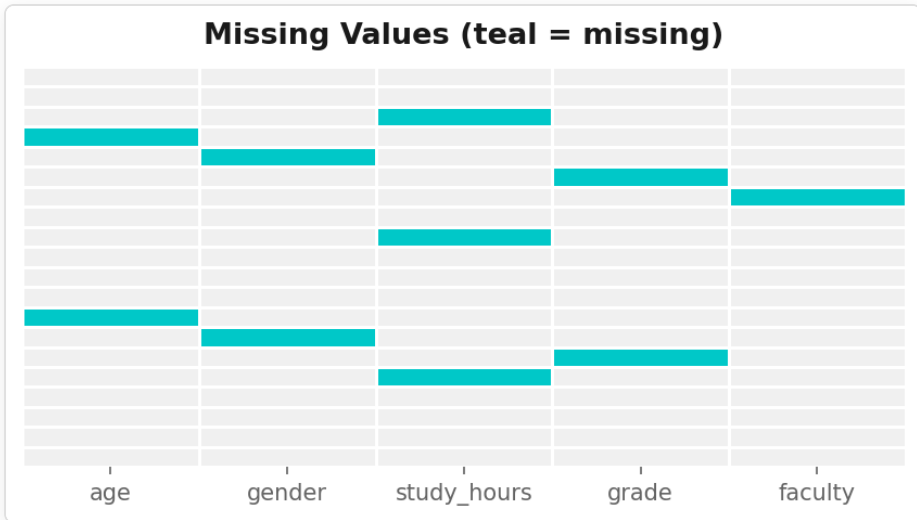
Garbage in — garbage out. This chapter = the data cleaning step.

What Can Go Wrong?

- Missing values (NaN , empty strings, -999)
- Wrong data types (age as string)
- Inconsistent categories ("MaLe" , "maLe" , "M")
- Outliers (age = 999 , salary = -1)
- Duplicate rows
- Mixed scales (salary in thousands vs. age in years)

Missing Values – Detection

```
df.isnull().sum()      # count per column
df.isnull().mean()     # proportion per column
```



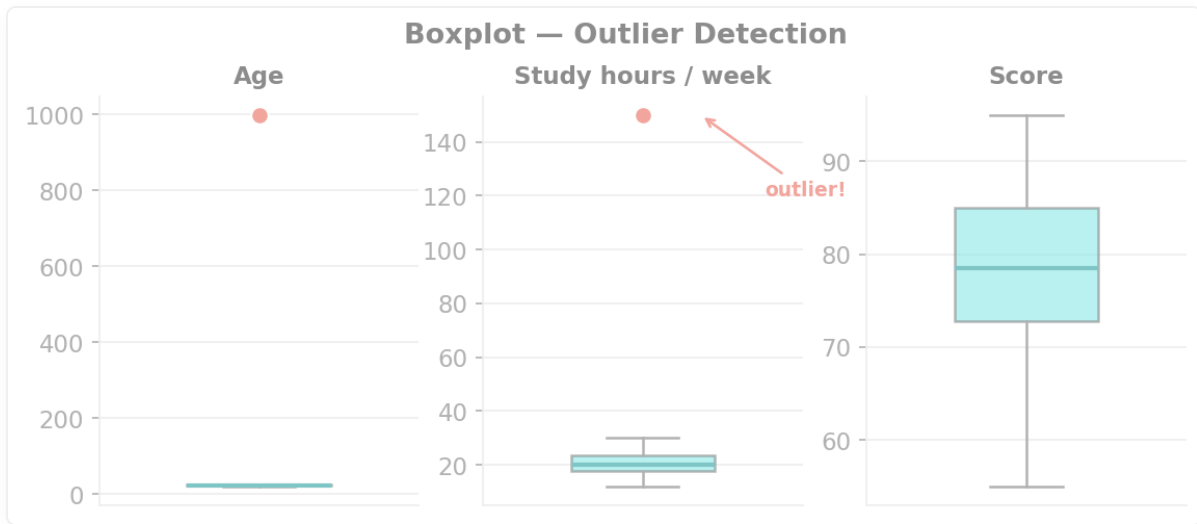
Rule of thumb: $< 5\%$ missing $\rightarrow$ drop rows $\cdot > 50\%$ in a column $\rightarrow$ drop column

Missing Values — Strategies

Strategy	When to use
Drop rows	Few missing, random missingness
Drop column	More than 50% missing
Fill with mean	Numerical, no outliers
Fill with median	Numerical, with outliers
Fill with mode	Categorical
Fill with constant	Domain knowledge (e.g. "Unknown")

 **Always impute AFTER train/test split** — otherwise data leakage!

Outliers



Detection: boxplot • IQR rule ($Q1 - 1.5 \cdot IQR$ / $Q3 + 1.5 \cdot IQR$) • Z-score $|z| > 3$

Treatment: Remove • Cap/clip • Log-transform • Keep (if real signal)

Feature Types

Type	Example	Ready for ML?
Numerical continuous	Age, salary	After scaling
Numerical discrete	# children	Usually yes
Categorical nominal	City, color	Need encoding
Categorical ordinal	S / M / L / XL	Need ordered encoding
Binary	Yes / No	Encode as 0 / 1

Encoding Categorical Features

One-Hot Encoding — faculty column

faculty	faculty_Arts	faculty_Engineering	faculty_Science
Science	0	0	1
Arts	1	0	0
Engineering	0	1	0
Arts	1	0	0
Science	0	0	1

```
pd.get_dummies(df, columns=['city']) # pandas  
# or: sklearn OneHotEncoder
```

⚠ With 100 cities → 100 new columns. Consider target encoding for high-cardinality features.

Feature Scaling



StandardScaler → mean=0, std=1 · **MinMaxScaler** → [0, 1]

Tree-based models (Decision Tree, Random Forest) → no scaling needed.

Train / Test Split

Train / Test Split (random_state=42, stratify=y)

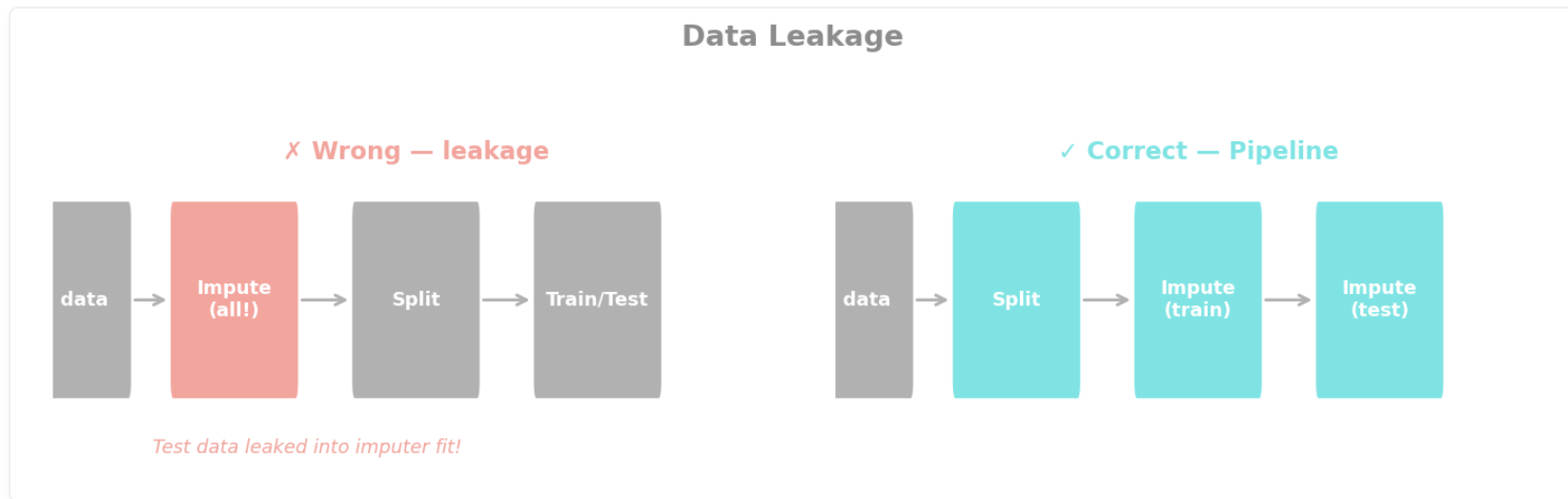
Full dataset (n = 20)

Train 80%

Test 20%

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Data Leakage ⚠



Fix: Use sklearn Pipelines — `fit()` only on train, `transform()` on both.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

The Preprocessing Checklist

- ✓ Inspect shape, dtypes, head
- ✓ Check missing values & outliers (visual inspection)
- ✓ Fix obvious errors (typos, wrong dtypes)
- ✓ Train / test split ← BEFORE any fitting!
- ✓ Impute missing values (fit on train, transform both)
- ✓ Detect and treat outliers (based on train statistics)
- ✓ Encode categorical features
- ✓ Scale numerical features (fit on train, transform both)

Golden rule: Everything that computes statistics (mean, median, IQR, scaler params) must be **fit on train only**.

Now: Exercises!

→ Open `03-exercises/ch02_data_cleaning_exercises.ipynb`

You will:

- Work with a messy dataset
- Apply all today's techniques step by step
- ~10 minutes

Key Takeaways

- Real data is always messy — cleaning is non-negotiable
- Impute missing values (after the split!)
- Encode all categories to numbers
- Scale features when using distance or gradient-based algorithms
- **Always split first, then preprocess**

Next: Chapter 3

Introduction to Supervised Learning

"Now that our data is clean, let's teach a machine to learn from it."